

EduSec Toolkit

Reporte técnico final — PIA PAC, Semestre Ene–Jun 2026
Grupo 01 · Josue Arcos · Johan Garay · Andrea Abundiz
Versión **1.0.0-final** · Mayo 2026

1. Resumen ejecutivo

EduSec Toolkit es una aplicación CLI escrita en C++17 que reúne seis módulos educativos de ciberseguridad — hashing (FNV-1a/32 y SHA-256), ataque por diccionario, enumeración de procesos, reconocimiento TCP con escaneo de los 20 servicios más comunes por defecto, captura pasiva de paquetes con raw sockets e inspección de mapas de memoria. El binario fue diseñado con un perfil estrictamente benigno: sin persistencia, sin exfiltración y sin capacidades destructivas, ejecutado únicamente dentro de máquinas virtuales aisladas controladas por el equipo.

El proyecto sirve como pieza educativa de doble propósito: por un lado ejercita la construcción modular y la programación de bajo nivel sobre POSIX/Linux; por otro, ofrece un objetivo realista para análisis estático, análisis dinámico e ingeniería inversa con Ghidra, Radare2, binutils, strace y ltrace. La compilación produce dos artefactos — uno con símbolos (576 KB) y otro stripped (68 KB) — para alimentar ambos enfoques de análisis. La implementación de SHA-256 se valida con el vector de prueba oficial de FIPS 180-4 §B.1.

2. Diseño

Arquitectura. Aplicación monolítica con un dispatcher mínimo en `src/main.cpp` que recibe `argv[1]` como subcomando, empaqueta el resto de argumentos en un `std::vector<std::string>` y delega a una función `run(args)` en el módulo correspondiente. Todos los módulos comparten esa firma para mantener un acoplamiento de interfaz uniforme y un grafo de llamadas predecible al hacer reversing.

```
main → switch(argv[1]) → modules/<name>::run(args)
■
■■■ hash_module (FNV + SHA-256)
■■■ bruteforce_module (dict attack)
■■■ proc_module (/proc enum)
■■■ netscan_module (TCP, top 20 def.)
■■■ sniffer_module (AF_PACKET raw)
■■■ mem_module (/proc maps)
```

Mapeo de subcomandos a técnicas.

Módulo	Subcomando	Técnica de ciberseguridad	Fase
sha256 + hash_module	hash	Hashing FNV-1a/32 y SHA-256 (FIPS 180-4)	II
bruteforce_module	brute	Ataque por diccionario contra FNV/SHA-256	II
proc_module	procs	Enumeración local de procesos via /proc	III
netscan_module	scan	TCP connect-scan (default: top 20 ports)	II
sniffer_module	sniff	Captura pasiva con AF_PACKET raw sockets	II
mem_module	mem	Lectura de /proc/<pid>/maps, RWX hunting	III

Decisiones técnicas.

Se descartaron OpenSSL y libpcap para reducir la superficie del binario y eliminar simbología externa que opacaría el análisis estático. SHA-256 se implementa íntegramente en sha256.{h,cpp} siguiendo FIPS 180-4 — sus constantes K[64] y H0[8] viven en .rodata y son una firma reconocible en el binario stripped, lo cual es *intencional* para el ejercicio de reversing. El netscan usa sockets no bloqueantes con select() para imponer timeouts cortos sin lanzar hilos. La compilación produce dos perfiles (-O0 -g y -O2 + strip) para que el mismo código fuente alimente análisis con símbolos y reversing ciego.

3. Implementación

El código fuente totaliza ~1 200 líneas de C++17 sin dependencias externas más allá de la libc y cabeceras POSIX/Linux. Los puntos técnicos más relevantes:

3.1. SHA-256 — núcleo criptográfico

Implementación incremental basada en un Context que expone update() y finalize(), permitiendo hashear archivos en bloques sin cargarlos en RAM. La compresión principal está en process_block():

```
for (int t = 0; t < 64; ++t) {
    uint32_t S1 = rotr(e, 6) ^ rotr(e, 11) ^ rotr(e, 25);
    uint32_t ch = (e & f) ^ (~e & g);
    uint32_t T1 = hh + S1 + ch + kk[t] + w[t];
    uint32_t S0 = rotr(a, 2) ^ rotr(a, 13) ^ rotr(a, 22);
    uint32_t maj = (a & b) ^ (a & c) ^ (b & c);
    uint32_t T2 = S0 + maj;
    hh=g; g=f; f=e; e=d+T1; d=c; c=b; b=a; a=T1+T2;
}
```

3.2. Bruteforce — diccionario

Streaming del archivo línea por línea (sin cargarlo a memoria), hashing de cada candidato con el algoritmo solicitado por --format, comparación case-insensitive del hex con el objetivo y reporte del tiempo total. Soporta --limit N para acotar diccionarios grandes.

3.3. Nmap — connect-scan no bloqueante

Cada puerto se prueba con: socket() → O_NONBLOCK → connect() → select() con timeout de 1 s → SO_ERROR. Si el usuario no pasa --ports, el módulo prueba automáticamente el **top 20 de servicios más comunes** (21, 22, 23, 25, 53, 80, 110, 111, 135, 139, 143, 443, 445, 993, 995, 1723, 3306, 3389, 5900, 8080), alineado con nmap --top-ports 20. La salida es una tabla limpia de dos columnas (PUERTO / ESTADO) para facilitar la lectura visual.

3.4. Sniffer — captura pasiva con AF_PACKET

socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL)) entrega tramas L2 completas. Se parsea Ethernet (14 B) → IPv4 (variable, ihl*4) → TCP/UDP. Si no se especifica --count, captura 20 paquetes por defecto. Cada IPv4 origen y destino observada se acumula en un std::set<std::string>; al finalizar la sesión, el sniffer imprime el conjunto de IPs únicas — útil para un vistazo rápido tipo "endpoints view" de Wireshark. No persiste nada a disco; la salida va exclusivamente a stdout.

3.5. Mem_module — /proc/<pid>/maps

Recorre el archivo línea a línea, extrae el rango addr_start-addr_end, permisos y ruta. Marca con [RWX] cualquier región r+w+x y suma un contador global. Adicionalmente lee /proc/<pid>/status para mostrar nombre, UID, VmPeak y VmRSS. **No** se accede a /proc/<pid>/mem: por diseño ético no se

lee la memoria del proceso, sólo su mapa.

4. Pruebas

Plan de pruebas reproducible en docs/tests.md con comandos exactos y outputs reales capturados durante la ejecución en la VM del equipo. Resumen de resultados:

Caso	Comando representativo	Resultado
Vector FIPS 180-4 §B.1	hash --format sha256 --string "abc"	ba7816bf...015ad ✓ coincide con NIST
Hash de archivo	hash --format sha256 --file makefile	f156629d...c448 (2921 B) ✓
FNV-1a/32	hash --format fnv --string "password"	0x364b5f18 ✓
Brute SHA-256	brute --format sha256 --hash <hex> --wordlist 3 palabras	brute MATCH en 2 intentos (0 ms) ✓
Brute FNV-1a/32	brute --format fnv --hash 0x364b5f18 --wordlist 3 palabras	brute MATCH en 2 intentos (0 ms) ✓
TCP scan top 20 (default)	scan --host 127.0.0.1	0 abierto(s) / 20 probado(s) ✓
TCP scan con puertos	scan --host 127.0.0.1 --ports 22,80,443	0 abierto(s) / 3 probado(s) ✓
Sniffer pasivo	sudo ./main sniff	20 paquetes parseados + resumen de IPs únicas ✓
Mem maps	mem --pid 1	regiones r-xp / rw-p identificadas ✓
Compilación estricta	make (-Wall -Wextra -Wpedantic)	0 warnings, 0 errores ✓

5. Análisis estático

Realizado con la suite de GNU binutils sobre ambos binarios y complementado con Ghidra/Radare2 (ver analysis/notes.md y capturas en evidence/). Hallazgos principales:

(a) Cabecera ELF. Tipo DYN (Position-Independent Executable) → ASLR habilitado. Clase ELF64, little-endian, x86-64, OS/ABI System V. Entry point del binario stripped: 0x48f0 (rutina _start de glibc).

(b) Tamaños y compresión. El stripping reduce el binario de 576 360 B a 67 888 B — una compresión del 88.2% — eliminando .symtab, .strtab y la información DWARF. El código en .text es idéntico entre ambos.

(c) Strings. strings -a -n 6 bin/main revela la superficie completa de subcomandos y flags (--format, --ports, --wordlist, --string, --file, --pid, --count, --limit). Esto se deja como demostración educativa: cualquier *payload* serio cifraría estos literales o los construiría dinámicamente.

(d) IoCs maliciosos. grep -iE 'http://[wget|nc -e|/bin/sh' sobre los strings devuelve **0 resultados**, coherente con el alcance benigno declarado.

(e) Símbolos identificados (sobre el binario con símbolos):

Offset	Símbolo	Función
0x51a4	main	Dispatcher
0x68d5	edusec::bruteforce_module::run	Diccionario
0x7921	edusec::hash_module::fnv1a_32	FNV core
0x8201	edusec::hash_module::run	Hash dispatcher
0x8a5b	edusec::mem_module::run	/proc maps

0xa544	edusec::netscan_module::run	TCP scan
0xc1dd	edusec::proc_module::run	Procs enum
0xcbc8	edusec::sha256::Context::process_block	SHA-256 core
0xe202	edusec::sniffer_module::run	Sniffer

Al stripear el binario estos nombres desaparecen, pero las firmas algorítmicas sobreviven: las 64 constantes K[t] de SHA-256 (0x428a2f98, 0x71374491, ...) y el par FNV (0x811C9DC5 / 0x01000193) permanecen en .rodata / inmediatos del .text y permiten reconocer cada rutina sin ayuda de la tabla de símbolos.

6. Análisis dinámico

Observación del comportamiento del binario en tiempo de ejecución con strace -f, ltrace, gdb y Wireshark (en paralelo al subcomando sniff).

6.1. Trazado de syscalls representativas

Con strace -f -e trace=network,desc ./main scan --host 127.0.0.1 --ports 22,80 se observan, por puerto:

```
socket(AF_INET, SOCK_STREAM, IPPROTO_IP) = 3
fcntl(3, F_GETFL) = 0x2 (flags O_RDWR)
fcntl(3, F_SETFL, O_RDWR|O_NONBLOCK) = 0
connect(3, {sa_family=AF_INET, sin_port=htons(22), ...}) = -1 EINPROGRESS
select(4, NULL, [3], NULL, {tv_sec=1}) = 1 (out [3])
getsockopt(3, SOL_SOCKET, SO_ERROR, ECONNREFUSED, [4]) = 0
close(3) = 0
```

Coherente con la lógica del módulo: connect() en O_NONBLOCK devuelve EINPROGRESS; select() espera escritura con timeout; SO_ERROR determina si el puerto está abierto o cerrado.

6.2. Sesión gdb sobre el binario con símbolos

Punto de quiebre en edusec::sha256::Context::process_block, se invoca ./main hash --format sha256 --string "abc" y se observa cómo w[0..15] recibe los 64 bytes de mensaje padded — en el caso de "abc" sólo el primer bloque — y cómo a..h evolucionan en las 64 rondas hasta sumarse a h_[0..7]. El digest final coincide bit a bit con el vector NIST.

6.3. Captura cruzada con Wireshark

Mientras se ejecuta sudo ./main sniff --count 20 en una ventana y ping -c 4 8.8.8.8 + curl example.com en otra, se compara la salida del sniffer con la captura simultánea de Wireshark. Cada paquete reportado por EduSec aparece en Wireshark con las mismas IPs, puertos y banderas TCP (S, A, P, F, R), validando el parser propio.

7. Ingeniería inversa

Se realizó un ejercicio de RE en dos pasadas para cuantificar cuánto se recupera con y sin símbolos.

7.1. Ground-truth con bin/main-debug

Importado en Ghidra, los nombres demangled aparecen directamente en el árbol de funciones (edusec::sha256::Context::process_block, etc.). El switch del dispatcher en main se ve como una

cadena de comparaciones strcmp contra los literales "hash", "procs", "scan", "brute", "sniff", "mem" — el grafo de llamadas reproduce literalmente el diseño documentado.

7.2. Reversing ciego sobre bin/main (stripped)

Sin símbolos, el reconocimiento se apoya en tres pivotes:

- **Constantes de SHA-256:** las 64 palabras de K[t] aparecen consecutivas en .rodata; la pareja H0[8] también es identificable. Cualquier función que lea esas constantes con un loop de 64 iteraciones es process_block.
- **Constantes de FNV-1a/32:** las inmediatos 0x811C9DC5 (offset) y 0x01000193 (prime) marcan la rutina fnv1a_32; el loop subyacente es hash ^= byte; hash *= prime.
- **Llamadas a syscalls:** socket(AF_PACKET, SOCK_RAW, htons(0x0003)) identifica sniffer_module::run; opendir("/proc") identifica proc_module; open("/proc/<pid>/maps") identifica mem_module.

El ejercicio confirma que el *stripping* sólo dificulta la lectura, no la impide. Para un atacante con familiaridad en constantes criptográficas estándar, recuperar la estructura del binario es cuestión de minutos.

8. Riesgos y mitigaciones

Riesgo	Mitigación implementada
Uso fuera de la VM (escaneo a redes ajenas)	VMs en host-only; aviso ético en README; no se setea setuid/setcap.
sniff requiere CAP_NET_RAW	Detección de error en socket() con mensaje claro; ejecución on-demand.
brute aplicado a hashes ajenos	Sólo opera contra hash/diccionario locales provistos por el operador.
Vulnerabilidades en parseo de argumentos	std::string/std::vector (no buffers crudos); validación 1..65535; compilación con -Wall -W
Lectura de /proc/<pid>/mem	No implementado por diseño; mem_module sólo lee maps y status.
Falsos positivos en marca [RWX]	Anotación textual; no acción automática; documentado en el reporte.
Filtración del binario	Sin persistencia, sin red saliente, sin destrucción; perfil benigno.
Reversing del binario stripped	Aceptado como ejercicio educativo; documentado en sección 7.

9. Conclusiones

EduSec Toolkit cumple los objetivos del PIA: ejercita la construcción modular de software en C++17 sobre cabeceras POSIX/Linux puras, integra seis técnicas de ciberseguridad funcionalmente verificadas (incluyendo una implementación criptográfica validada contra el vector oficial de NIST) y entrega dos binarios para alimentar simultáneamente análisis con símbolos y reversing *ciego*.

Los hallazgos del análisis cuantifican algo que en clase suele presentarse como afirmación: el *stripping* reduce 88% el tamaño del binario pero no oculta el algoritmo cuando éste usa constantes estándar. Esto refuerza la lección de por qué los *payloads* reales emplean ofuscación de literales y por qué los hashes para contraseñas requieren sal y funciones lentas como bcrypt o Argon2 — un FNV-1a/32 sin sal cae en 0 ms con un diccionario trivial; SHA-256 sin sal sigue siendo recuperable con wordlists comunes en CPU.

Trabajo futuro identificado: endurecimiento del binario con `-fstack-protector-strong`, `-fcf-protection=full`, `-D_FORTIFY_SOURCE=2` y `-Wl,-z,now -Wl,-z,relro`; filtros BPF para reducir ruido en sniffer; reintroducción opcional de banner-grabbing como subcomando dedicado (banner) para mantener limpia la tabla de scan; benchmark FNV vs SHA-256 en H/s contra wordlists largas; y migración opcional a CMake.

Aviso final. Este trabajo fue desarrollado y probado bajo consentimiento académico exclusivamente para fines educativos. Su redistribución y uso quedan restringidos al contexto del curso PAC del Semestre Enero–Junio 2026.